

EXAMFORGE AI

Enterprise Migration Architecture Specification

Flutter to Next.js 16 — Complete Architecture Design

Prepared by: Lead Enterprise Software Architect & Senior Migration Engineer

Date: August 2026 | Classification: CONFIDENTIAL

Version 1.0 — Phase 1 Deliverable

30 Architecture Sections	80+ Screens Mapped	130+ Routes Documented	20+ Feature Modules	100+ Providers Analyzed
--------------------------------	--------------------------	------------------------------	---------------------------	-------------------------------

TABLE OF CONTENTS

1	Source Architecture Overview (Flutter)
2	Target Architecture Overview (Next.js 16)
3	Folder Structure
4	App Router Structure & Route Mapping
5	Layout Hierarchy
6	Authentication Architecture
7	Middleware & RBAC
8	Zustand Store Architecture
9	TanStack Query Architecture
10	API Layer & Supabase Client
11	Server Actions & Server/Client Components
12	Realtime Architecture
13	Offline & IndexedDB Architecture
14	Theme System & Design System
15	shadcn/ui Component Mapping
16	Tailwind CSS Architecture
17	Form & Validation Architecture
18	Error Boundary, Loading & Suspense Strategy
19	Feature Module Organization
20	Environment Variables & CI/CD
21	Testing Architecture
22	Architecture Diagrams
23	Migration Checklist

1. Source Architecture Overview (Flutter)

The ExamForge AI Flutter application follows a **Clean Architecture** pattern organized into feature-based modules under the `lib/features/` directory. Each feature module adheres to a strict three-layer separation: **Domain** (entities, repository interfaces, use cases), **Data** (models, datasources, repository implementations), and **Presentation** (providers, widgets, pages). This pattern is consistently applied across all 20+ feature modules, providing a clear separation of concerns that maps well to the target Next.js architecture.

The application uses **Riverpod** (flutter_riverpod v2.5.1) for state management with a centralized dependency injection container in `config/dependency_injection.dart` that registers 100+ providers. Navigation is handled by **GoRouter** (v14.2.0) with declarative route definitions, `ShellRoute` for persistent layouts, and a guard-based redirect system for authentication, onboarding, and role-based access control. The backend is powered entirely by **Supabase** (v2.5.6) leveraging Auth, PostgreSQL, Realtime, Storage, and Edge Functions.

1.1 Architecture Pattern Summary

Pattern	Flutter Implementation	Next.js Target
State Management	Riverpod (StateNotifier, FutureProvider, StreamProvider)	Zustand + TanStack Query
Navigation	GoRouter with ShellRoute + redirect guards	Next.js App Router + middleware.ts
Backend	Supabase Flutter SDK (Auth, DB, Realtime, Storage)	Supabase JS SDK v2 + Server Actions
DI	Centralized Riverpod provider container	Module-level barrel exports + context providers
Forms	Formz (EmailInput, PasswordInput, etc.)	React Hook Form + Zod schemas
Offline	Drift (SQLite) local database	IndexedDB via Dexie.js
Notifications	Firebase Cloud Messaging + Supabase tokens	Web Push API + Supabase Realtime
Payments	Flutterwave (via Edge Functions)	Flutterwave (via Edge Functions, unchanged)
AI	Multi-provider registry (OpenAI, Gemini)	Same Edge Functions (ai-complete, ai-stream)
Error Handling	Sealed Result<T> + Failure variants	Result pattern + Error Boundaries
Theming	Material 3 + ThemeNotifier (Riverpod)	Tailwind CSS + next-themes + CSS vars
Responsive	flutter_screenutil + responsive_framework	Tailwind responsive breakpoints
Accessibility	Accessible widgets + accessibility_framework	Radix UI primitives + ARIA
Observability	Custom crash reporter, metrics, tracing	Sentry + Vercel Analytics + OpenTelemetry
Resilience	Circuit breaker, timeout, graceful degradation	Same patterns in TypeScript

1.2 Source Directory Structure

```
lib/  
main.dart # Bootstrap & entry point  
app.dart # Root ExamForgeApp widget  
config/ # Configuration & DI  
app_config.dart # Feature flags, timeouts, environment  
env_config.dart # .env loader & env vars  
supabase_config.dart # Supabase client init & realtime  
dependency_injection.dart # Central DI container (100+ providers)
```

```
routing/ # Navigation
app_router.dart # GoRouter with ALL routes
route_guards.dart # Auth, Onboarding, RoleBased guards
route_names.dart # 130+ route path constants
core/ # Shared infrastructure
themes/ # M3 ThemeData (light+dark)
constants/ # App & API constants
errors/ # Sealed Failure + Exception types
utils/ # Result<T>, logger, validators
network/ # Dio API client, connectivity
storage/ # Drift offline DB, cache manager
security/ # Encryption, admin & AI security
observability/ # Crash reporter, metrics, tracing
performance/ # Perf manager, AI cache
resilience/ # Circuit breaker, timeout, degradation
accessibility/ # Accessible widgets framework
sync/ # Sync engine
connectivity/ # Connectivity engine
services/ # Cross-cutting services
auth_service.dart # Supabase Auth wrapper
storage_service.dart # SecureStorage + SharedPrefs
notification_service.dart # FCM + Supabase token sync
ai/ # AI service, provider registry, prompt engine
cbt/ # Anti-cheat, timer, realtime, auto-save
results/ # AI grading, analytics, report gen
shared/ # Shared models, providers, widgets
models/user_role.dart # UserRole enum + extensions
providers/auth_state_provider.dart # Central auth state
widgets/ # 13 reusable UI components
features/ # Feature modules (Clean Architecture)
auth/ # 5 pages, 5 use cases
dashboard/ # Role-specific dashboards (4 roles)
question_bank/ # 8 pages, 11 use cases
ai_generator/ # 7 pages, 9 use cases
cbt_engine/ # 8+ pages, 12 use cases
billing/ # 11 pages, 11 use cases
marketplace/ # 14 pages, 50+ use cases
school_management/ # 30+ pages, 9 use case sets
teacher_workspace/ # 16 pages, 50+ use cases
student_portal/ # 11 pages
parent_portal/ # 11 pages, 15 use cases
super_admin/ # 12 pages
communication/ # 13 pages, 30+ use cases
+ 6 more feature modules
```

2. Target Architecture Overview (Next.js 16)

The target architecture leverages **Next.js 16** with the **App Router** paradigm, fully embracing React Server Components (RSC), Server Actions, and Middleware for a performant, secure, and maintainable application. The architecture follows a feature-based module organization that mirrors the Flutter Clean Architecture structure, while leveraging Next.js conventions for colocation of server and client logic within each route segment.

Key architectural principles include: **Server-First Rendering** — default to Server Components, opt into client interactivity only when needed; **Progressive Enhancement** — core content renders without JavaScript, interactivity layers on top; **Edge-Optimized** — middleware runs at the edge for auth checks and redirects; **Type Safety Throughout** — TypeScript strict mode with Zod runtime validation at every API boundary; **Offline Resilience** — IndexedDB-backed offline storage with background sync for exam-taking scenarios.

2.1 Technology Stack

Category	Technology	Version	Purpose
Framework	Next.js	16	App Router, RSC, Server Actions, Middleware
Language	TypeScript	5.x	Strict mode, path aliases, branded types
UI Library	React	19	Server Components, Suspense, use() hook
Styling	Tailwind CSS	4	Utility-first, CSS custom properties, dark mode
Components	shadcn/ui	latest	Radix primitives + Tailwind styled, copy-owned
Animation	Framer Motion	11+	Layout animations, page transitions, gestures
State	Zustand	5	Client-side global state (auth, UI, preferences)
Data Fetching	TanStack Query	5	Server state, caching, invalidation, pagination
Forms	React Hook Form	7	Performant forms with controlled/uncontrolled modes
Validation	Zod	3	Schema validation, type inference, form integration
Backend	Supabase JS SDK	2	Auth, DB, Realtime, Storage, Edge Functions
Offline DB	Dexie.js	4	IndexedDB wrapper for offline exam storage
Icons	Lucide React	latest	Tree-shakeable icon set (replaces Iconsax)
Date	date-fns	3	Lightweight date formatting & manipulation
Charts	Recharts	2	Composable chart components for analytics
PDF	@react-pdf/renderer	4	Client-side report generation
Testing	Vitest + Playwright	latest	Unit + E2E testing

2.2 Architectural Principles

- **Server Component Default:** Every component is a Server Component unless it requires interactivity (event handlers, hooks, browser APIs). This minimizes client bundle size and maximizes SEO.
- **Colocation:** Route-specific components, actions, and utilities live alongside their page.tsx. Shared code lives in lib/ and components/ at the root level.
- **Data Fetching at the Route Level:** Server Components fetch data directly via Supabase client. Client components use TanStack Query for mutations and realtime subscriptions.
- **Progressive Enhancement:** Core functionality (viewing exams, reading content) works without JS. Interactive features (taking exams, AI generation) require client-side hydration.
- **Type-Safe API Boundaries:** Every Server Action validates input with Zod. Every Supabase query is typed with generated types from supabase gen types.
- **Feature Encapsulation:** Each feature module exports a public API surface. Internal implementation details are not importable from other features.

3. Folder Structure

The Next.js project follows a feature-based organization that maps directly from the Flutter Clean Architecture modules. The `src/` directory contains all application code, with the App Router structure in `src/app/` and feature modules in `src/features/`. Each feature module maintains its

domain/data/presentation layers, preserving the Clean Architecture discipline from Flutter.

```
examforge-ai/  
src/  
app/ # Next.js App Router  
(public)/ # Route group: unauthenticated  
login/page.tsx  
register/page.tsx  
forgot-password/page.tsx  
reset-password/page.tsx  
verify-email/page.tsx  
(authenticated)/ # Route group: authenticated (middleware guard)  
dashboard/  
page.tsx # Role-based redirect  
teacher/page.tsx  
student/page.tsx  
school-admin/page.tsx  
super-admin/page.tsx  
profile/page.tsx  
settings/page.tsx  
notifications/page.tsx  
question-bank/  
page.tsx # Dashboard  
list/page.tsx  
create/page.tsx  
[id]/page.tsx # Detail + Edit  
import/page.tsx  
export/page.tsx  
collections/page.tsx  
ai-generator/  
generate/page.tsx  
review/page.tsx  
improve/page.tsx  
document/page.tsx  
history/page.tsx  
prompts/page.tsx  
exams/  
page.tsx # List  
create/page.tsx  
[id]/page.tsx # Detail  
[id]/edit/page.tsx  
[id]/monitor/page.tsx  
[id]/take/page.tsx # Client Component (heavy)  
[id]/results/page.tsx  
templates/page.tsx  
my-exams/page.tsx  
billing/  
plans/page.tsx  
checkout/page.tsx  
callback/page.tsx  
history/page.tsx  
invoices/[id]/page.tsx  
ai-credits/page.tsx  
coupons/page.tsx  
referrals/page.tsx  
licenses/page.tsx  
revenue/page.tsx  
marketplace/  
page.tsx # Home  
search/page.tsx  
[productId]/page.tsx  
cart/page.tsx  
seller/page.tsx  
super-admin/ # SuperAdmin only (middleware guard)  
schools/page.tsx
```

```
users/page.tsx
ai/page.tsx
billing/page.tsx
support/page.tsx
security/page.tsx
infrastructure/page.tsx
analytics/page.tsx
settings/page.tsx
teacher/ # Teacher workspace
lesson-plans/page.tsx
worksheets/page.tsx
assignments/page.tsx
rubrics/page.tsx
resources/page.tsx
content-assistant/page.tsx
scheme-of-work/page.tsx
student/ # Student portal
assignments/page.tsx
flashcards/page.tsx
practice/page.tsx
goals/page.tsx
study-planner/page.tsx
ai-tutor/page.tsx
parent/ # Parent portal
children/[id]/page.tsx
messaging/page.tsx
calendar/page.tsx
insights/page.tsx
reports/page.tsx
communication/
page.tsx # Conversations list
[conversationId]/page.tsx
forums/page.tsx
forums/[forumId]/page.tsx
announcements/page.tsx
calendar/page.tsx
analytics/page.tsx
results/
[examId]/page.tsx
student/[studentId]/page.tsx
admin/page.tsx
layout.tsx # Root layout (ThemeProvider, fonts)
loading.tsx # Global loading skeleton
not-found.tsx # 404 page
error.tsx # Global error boundary
forbidden.tsx # 403 page
unauthorized.tsx # 401 page
components/ # Shared UI components
ui/ # shadcn/ui components (copy-owned)
button.tsx, card.tsx, dialog.tsx, input.tsx, ...
layout/ # Layout components
app-shell.tsx # Authenticated shell (sidebar, header)
sidebar.tsx, header.tsx, mobile-nav.tsx
forms/ # Form field components
text-field.tsx, select-field.tsx, date-field.tsx
data/ # Data display components
data-table.tsx, stat-card.tsx, empty-state.tsx, error-state.tsx
feedback/ # Feedback components
loading-skeleton.tsx, toast.tsx, progress.tsx
lib/ # Shared utilities
supabase/
client.ts # Browser Supabase client
server.ts # Server Supabase client
admin.ts # Service role client (Edge Functions only)
middleware.ts # Middleware client for auth
```

```
types.ts # Generated database types
auth/
config.ts # Auth config, redirects, role mapping
permissions.ts # Permission definitions per role
session.ts # Session helpers
api/
client.ts # Typed API client wrapper
errors.ts # API error classes
result.ts # Result<T, E> type
db/
indexddb.ts # Dexie.js database schema
sync.ts # Sync engine
migrations.ts # IndexedDB migration runner
utils/
cn.ts, format.ts, validate.ts, logger.ts
constants/
routes.ts # Route constants
storage-keys.ts # LocalStorage/IndexedDB keys
limits.ts # Rate limits, page sizes, thresholds
hooks/
use-auth.ts, use-role.ts, use-realtime.ts, use-debounce.ts
use-media-query.ts, use-offline.ts, use-intersection.ts
stores/ # Zustand stores
auth-store.ts, ui-store.ts, theme-store.ts, exam-store.ts
validators/ # Zod schemas
auth.ts, question.ts, exam.ts, billing.ts, user.ts
features/ # Feature modules
auth/
actions/ # Server Actions
login.action.ts, signup.action.ts, logout.action.ts
components/ # Feature-specific components
login-form.tsx, register-form.tsx, password-strength.tsx
hooks/ # Feature-specific hooks
types/ # Feature-specific types
validators/ # Feature-specific Zod schemas
index.ts # Public API barrel export
question-bank/
actions/ # CRUD Server Actions
components/ # QuestionCard, QuestionEditor, etc.
hooks/ # useQuestions, useQuestionFilters
types/
index.ts
cbt-engine/
actions/
components/
hooks/ # useExamTimer, useAntiCheat, useAutoSave
lib/ # exam-timer.ts, anti-cheat.ts, session-recovery.ts
types/
index.ts
billing/
marketplace/
school-management/
teacher-workspace/
student-portal/
parent-portal/
super-admin/
communication/
analytics/
ai-generator/
ai-coach/
results/
middleware.ts # Root middleware (auth + RBAC + redirects)
public/
fonts/inter/ # Self-hosted Inter font files
images/ # Static images
```



```
icons/ # App icons, favicon, manifest
supabase/
functions/ # Edge Functions (unchanged from Flutter)
migrations/ # Database migrations
config.toml # Supabase project config
tests/
e2e/ # Playwright E2E tests
unit/ # Vitest unit tests
integration/ # Integration tests
.env.local # Environment variables (local)
.env.production # Environment variables (production)
next.config.ts # Next.js configuration
tailwind.config.ts # Tailwind configuration
tsconfig.json # TypeScript configuration
vitest.config.ts # Test configuration
playwright.config.ts # E2E test configuration
```

4. App Router Structure & Route Mapping

The Next.js App Router replaces GoRouter with a file-system-based routing paradigm. Each Flutter GoRoute maps to a Next.js route segment (directory with page.tsx). Route groups using parentheses (**public**) and (**authenticated**) replace GoRouter ShellRoute, providing distinct layouts for authenticated and unauthenticated contexts. Dynamic routes use bracket notation [**id**] for path parameters, replacing GoRouter path parameters. Middleware handles all guard logic previously implemented in GoRouter redirects.

4.1 Route Group Architecture

Route Group	Layout	Middleware Guard	Flutter Equivalent
(public)	PublicLayout — minimal, centered card	Redirect to /dashboard if authenticated	GoRouter top-level routes
(authenticated)	AppShell — sidebar, header, nav	Redirect to /login if unauthenticated	GoRouter ShellRoute
(authenticated)/super-admin/*	Inherits AppShell	Role check: superAdmin only	RoleBasedGuard
(authenticated)/teacher/*	Inherits AppShell	Role check: teacher+	RoleBasedGuard
(authenticated)/student/*	Inherits AppShell	Role check: student+	RoleBasedGuard
(authenticated)/parent/*	Inherits AppShell	Role check: parent+	RoleBasedGuard

4.2 Complete Route Mapping

Next.js Route	Page Component	Group	Rendering	Flutter Route
/	Splash/Rredirect	(public)	Server Component	/splash
/login	LoginPage	(public)	Client Component	/login
/register	RegisterPage	(public)	Client Component	/register
/forgot-password	ForgotPasswordPage	(public)	Client Component	/forgot-password

Next.js Route	Page Component	Group	Rendering	Flutter Route
/reset-password	ResetPasswordPage	(public)	Client Component	/reset-password
/verify-email	VerifyEmailPage	(public)	Client Component	/verify-email
/dashboard	DashboardRedirector	(authenticated)	Server Component	/dashboard
/dashboard/teacher	TeacherDashboard	(authenticated)	Server Component	/dashboard/teacher
/dashboard/student	StudentDashboard	(authenticated)	Server Component	/dashboard/student
/dashboard/school-admin	SchoolAdminDashboard	(authenticated)	Server Component	/dashboard/school-admin
/dashboard/super-admin	SuperAdminDashboard	(authenticated)	Server Component	/dashboard/super-admin
/question-bank	QuestionBankDashboard	(authenticated)	Server Component	/question-bank
/question-bank/list	QuestionListPage	(authenticated)	Server + Client	/question-bank/list
/question-bank/create	QuestionEditorPage	(authenticated)	Client Component	/question-bank/create
/question-bank/[id]	QuestionDetailPage	(authenticated)	Server Component	/question-bank/detail
/exams	ExamListPage	(authenticated)	Server Component	/exams
/exams/create	ExamBuilderPage	(authenticated)	Client Component	/exams/create
/exams/[id]/take	ExamTakePage	(authenticated)	Client Component (heavy)	/exams/take
/exams/[id]/monitor	ExamMonitorPage	(authenticated)	Client + Realtime	/exams/monitor
/billing/plans	SubscriptionPlansPage	(authenticated)	Server Component	/billing/subscription-plans
/billing/checkout	CheckoutPage	(authenticated)	Client Component	/billing/checkout
/billing/callback	PaymentCallbackPage	(authenticated)	Server Component	/billing/payment-callback
/marketplace	MarketplaceHome	(authenticated)	Server Component	/marketplace
/super-admin/*	SuperAdminPages (12)	(authenticated)	Server + Client	/super-admin/*
/teacher/*	TeacherWorkspace (16)	(authenticated)	Server + Client	/teacher-workspace/*
/student/*	StudentPortal (11)	(authenticated)	Server + Client	/student-portal/*
/parent/*	ParentPortal (11)	(authenticated)	Server + Client	/parent-portal/*
/communication/*	Communication (13)	(authenticated)	Client + Realtime	/communication/*

5. Layout Hierarchy

The layout hierarchy replaces Flutter's `ShellRoute` and `MaterialApp` structure with Next.js nested layouts. Each `layout.tsx` wraps its children, providing shared UI elements (navigation, headers, providers) that persist across route transitions. This eliminates the need for GoRouter `ShellRoute` and provides a more composable approach to shared layout concerns. Layouts are Server Components by default, with Client Component boundaries only where interactivity is required (sidebar state, theme toggle, notification bell).

5.1 Layout Nesting Tree

```
app/layout.tsx # Root layout
ThemeProvider (next-themes)
SupabaseProvider (client)
QueryClientProvider (TanStack)
Toaster (sonner)
Inter font + CSS variables
app/(public)/layout.tsx # Public layout
Centered card container
Background gradient/pattern
No navigation chrome
app/(authenticated)/layout.tsx # Authenticated layout (AppShell)
Sidebar (collapsible, role-aware)
Header (user menu, notifications, search)
MobileNav (bottom sheet on mobile)
RoleProvider (Zustand context)
OfflineIndicator
RealtimeProvider (exam subscriptions)
```

5.2 Layout Component Mapping

Flutter Widget	Next.js Component	Type	Notes
ExamForgeApp (MaterialApp.router)	app/layout.tsx	Server	Root providers, fonts, metadata
ShellRoute (BottomNav + AppBar)	(authenticated)/layout.tsx	Server	Sidebar + Header shell
AppBottomNav	MobileNav (Sheet)	Client	Bottom sheet on mobile, hidden on desktop
AppAppBar	Header component	Client	User menu, notifications, search, theme toggle
AppNavigationDrawer	Sidebar component	Client	Collapsible, role-filtered nav items
DashboardShell	dashboard/layout.tsx	Server	Tab navigation for dashboard sub-routes
OnboardingPage (PageView)	Onboarding carousel	Client	Framer Motion animations

6. Authentication Architecture

The authentication system maps from Flutter's `AuthService` + Supabase Auth SDK to Next.js's Supabase Auth helpers with middleware-based session management. The key architectural shift is moving

session validation from GoRouter redirect guards to Next.js Middleware, which executes at the edge before any rendering occurs. This eliminates the client-side redirect flash that occurs in the Flutter implementation and provides instant auth state resolution.

6.1 Auth Flow Comparison

Concern	Flutter Implementation	Next.js Implementation
Session storage	Supabase Auth + FlutterSecureStorage	Supabase Auth + httpOnly cookies (SSR)
Session check	GoRouter redirect on navigation	Middleware.ts runs at edge before render
Login	AuthNotifier.login() -> Supabase signInWithPassword	Server Action -> Supabase signInWithPassword -> set cookie
Signup	AuthNotifier.signUp() -> role forced to student	Server Action -> role forced to student -> set cookie
Logout	Supabase signOut() + clearSecureStorage	Server Action -> Supabase signOut() + clear cookie
Password reset	Supabase resetPasswordForEmail (deep link)	Supabase resetPasswordForEmail (email link to /reset-password)
Email verification	Check session.emailConfirmedAt	Middleware redirects unverified to /verify-email
Magic link	Supabase signInWithOtp	Supabase signInWithOtp (email link to /auth/callback)
Token refresh	Supabase auto-refresh (10s timeout)	Supabase auto-refresh + middleware cookie sync
Role resolution	userRoleProvider (FutureProvider) from SecureStorage	JWT app_metadata.role from cookie session
Onboarding guard	onboardingCompleteProvider (FutureProvider)	Middleware checks profile.onboarding_complete column

6.2 Auth Client Architecture

The Supabase client is instantiated in three contexts: **Server** (for Server Components and Server Actions), **Browser** (for Client Components and TanStack Query), and **Middleware** (for session validation at the edge). The server client uses `createServerClient` with cookie-based auth, the browser client uses `createBrowserClient` with the singleton pattern, and the middleware client uses `createMiddlewareClient` for session refresh and cookie management.

Client Type	File	Auth Method	Use In	Singleton
Server	lib/supabase/server.ts	Cookies (httpOnly)	Server Components, Server Actions	Per-request
Browser	lib/supabase/client.ts	Auto from cookie	Client Components, TanStack Query	Yes (singleton)
Middlewar e	lib/supabase/middlewar e.ts	Cookies (read/refresh/set)	middleware.ts only	Per-request
Admin	lib/supabase/admin.ts	Service role key	Edge Functions only (never client)	N/A

6.3 Security Hardening

- **httpOnly cookies:** Session tokens stored in httpOnly, secure, SameSite=Lax cookies — inaccessible to XSS, unlike Flutter's FlutterSecureStorage which is accessible to privileged client code.
- **PKCE flow:** Supabase Auth configured with PKCE (Proof Key for Code Exchange) for the authorization code flow, matching the Flutter implementation's PKCE configuration.
- **Role enforcement at signup:** The Server Action for signup always overrides the role claim to "student" in user_metadata, mirroring the Flutter AuthRemoteDataSourceImpl.signup() security constraint.
- **Middleware-first auth:** All auth checks happen in middleware.ts before any page renders. No client-side redirects for protected routes — eliminates the loading flash present in Flutter's GoRouter redirect approach.
- **Session refresh:** Middleware automatically refreshes expired sessions and updates cookies on every request, ensuring seamless session continuity without user intervention.

7. Middleware & RBAC

The middleware.ts file replaces all three Flutter route guards (AuthGuard, OnboardingGuard, RoleBasedGuard) with a single, sequential middleware chain that executes at the edge. This provides instant redirects without client-side JavaScript, eliminating the loading flash that occurs when GoRouter evaluates redirect functions. The middleware chain processes requests in priority order: authentication check, onboarding check, then role-based access control — matching the Flutter guard evaluation order exactly.

7.1 Middleware Chain

Prio rity	Guard	Condition	Action	Flutter Equivalent
1	AuthGuard	No valid session	Redirect to /login	AuthGuard redirect
2	AuthGuard (reverse)	Valid session on /login, /register	Redirect to /dashboard	AuthGuard reverse redirect
3	OnboardingGuard	Session valid but onboarding incomplete	Redirect to /onboarding	OnboardingGuard redirect
4	RoleBasedGuard	Session valid but role not in route's allowedRoles	Redirect to user's dashboard	RoleBasedGuard redirect
5	EmailVerification	Session valid but email not verified (protected routes only)	Redirect to /verify-email	Check in AuthNotifier

7.2 RBAC Role Definitions

Role	privile geLev el	Dashboard Route	Allowed Route Prefixes	Flutter UserRole
superAd min	4	/dashboard/supe r-admin	All routes	UserRole.superAdmin
schoolAd min	3	/dashboard/scho ol-admin	/dashboard, /question-bank, /exams, /billing, /school-management, /communication, /analytics	UserRole.schoolAdmin
teacher	2	/dashboard/teac her	/dashboard, /question-bank, /exams, /ai-generator, /teacher, /communication, /results	UserRole.teacher

Role	privilegeLevel	Dashboard Route	Allowed Route Prefixes	Flutter UserRole
student	1	/dashboard/student	/dashboard, /exams/take, /student, /marketplace, /billing	UserRole.student
parent	1	/dashboard/parent	/dashboard, /parent, /communication	UserRole.parent

7.3 Route Permission Configuration

Each route segment can declare its required roles via a `route-config.ts` export or via the middleware's route-to-role mapping. The middleware reads the user's role from the JWT `app_metadata` (populated by Supabase Auth), eliminating the need for a separate database query or FutureProvider resolution that the Flutter implementation requires. This makes role-based redirects instant — they occur at the edge before any server-side rendering begins.

8. Zustand Store Architecture

Zustand replaces Riverpod for client-side global state management. Unlike Riverpod's provider-based DI system with 100+ providers, Zustand uses a minimal number of focused stores that group related state. Each store is a standalone TypeScript module with explicit actions and selectors. Server state (API data) is managed by TanStack Query rather than Zustand — Zustand handles only UI state, auth state, and client-side ephemeral data that does not originate from API calls.

8.1 Store Inventory

Store	State Shape	Flutter Providers Replaced	Persistence
auth-store	user, role, isAuthenticated, isEmailVerified, session	authStateProvider, currentUserProvider, isAuthenticatedProvider, userRoleProvider, resolvedUserRoleProvider, + 8 convenience providers	Cookie (httpOnly) + Zustand persist (role cache)
ui-store	sidebarOpen, activeModule, searchQuery, commandPaletteOpen, toasts	No direct equivalent (scattered StateNotifiers)	SessionStorage (sidebar state)
theme-store	themeMode, seedColor, fontSize, reducedMotion	themeProvider, ThemeNotifier	LocalStorage (next-themes)
exam-session-store	activeExamId, answers, currentQuestionIndex, timerState, isOffline, syncStatus	9 CBT providers (examBuilder, examTaker, etc.)	IndexedDB (auto-save exam progress)
notification-store	unreadCount, toasts, permissionStatus	notificationServiceProvider (partial)	None (ephemeral)
onboarding-store	step, isComplete, skipOnboarding	onboardingCompleteProvider, onboardingProvider	LocalStorage

8.2 Store Design Principles

- **Minimal surface area:** Each store exports exactly one hook (useAuthStore, useUIStore, etc.). Internal state is not directly accessible — consumers use selector functions to subscribe to specific state slices, preventing unnecessary re-renders.
 - **No server state duplication:** Zustand stores never cache API responses. All server-derived data (questions, exams, users, billing) is managed by TanStack Query. Zustand stores only hold UI state and auth session data that must be synchronously accessible without network requests.
 - **Devtools integration:** All stores are wrapped with Zustand devtools in development mode, providing Redux DevTools-compatible inspection of state changes and action history.
 - **Hydration safety:** Stores that persist to localStorage/sessionStorage use a hydration pattern that avoids mismatches between server and client renders. The theme store leverages next-themes for SSR-safe theme resolution.
 - **Testability:** Each store exposes a reset() method for test isolation. Stores are plain functions with no React dependency, enabling testing without rendering components.
-

9. TanStack Query Architecture

TanStack Query replaces the majority of Riverpod FutureProvider and StreamProvider usage for server state management. It provides automatic caching, background refetching, stale-while-revalidate semantics, pagination, infinite scrolling, optimistic updates, and mutation handling — all of which were manually implemented in the Flutter Riverpod providers. The query client is provided at the root layout level and configured with feature-specific defaults for stale times and cache times.

9.1 Query Key Architecture

Query keys follow a hierarchical tuple structure that enables precise invalidation. Each feature defines its key factory in a dedicated file, ensuring type safety and preventing key collisions across features. The key hierarchy supports list filtering, individual item access, and related data queries.

Feature	Key Factory	Example Keys	Stale Time	Cache Time
auth	['auth', 'session']	['auth', 'session'], ['auth', 'profile']	5 min	30 min
questions	['questions', filters]	['questions', {subject:'math'}], ['questions', 'detail', id]	2 min	10 min
exams	['exams', filters]	['exams', {status:'active'}], ['exams', 'detail', id]	1 min	5 min
billing	['billing', type]	['billing', 'subscription'], ['billing', 'invoices']	5 min	30 min
marketplace	['marketplace', filters]	['marketplace', {category:'math'}], ['marketplace', 'product', id]	2 min	15 min
schools	['schools', filters]	['schools'], ['schools', 'detail', id]	5 min	30 min
communication	['communication', type]	['communication', 'conversations'], ['communication', 'messages', id]	30 sec	5 min
analytics	['analytics', type, range]	['analytics', 'platform', {from, to}]	10 min	60 min

9.2 Mutation Pattern

All mutations (create, update, delete) follow a consistent pattern: the Server Action performs the Supabase mutation with Zod-validated input, returns a typed result, and the TanStack Query mutation's onSuccess callback invalidates related query keys to trigger background refetches. For critical mutations (exam answer submission, payment processing), optimistic updates are applied to the cache immediately, with automatic rollback on error.

9.3 Prefetching Strategy

- **Route-level prefetching:** Server Components prefetch data during SSR using the Supabase server client. The data is dehydrated into the TanStack Query cache via the HydrationBoundary component, providing instant initial render with zero loading states for the first page load.
- **Link hover prefetching:** The custom Link component prefetches the target route's queries on mouseenter using queryClient.prefetchQuery(), providing instant navigation without loading spinners.
- **Background refetching:** Stale queries are refetched in the background on window focus (desktop) and on route change (mobile), ensuring data freshness without disrupting the user experience.
- **Infinite scroll:** List pages (question bank, exams, marketplace) use TanStack Query's useInfiniteQuery with cursor-based pagination, replacing the PaginatedQueryMixin from Flutter's core/network/ module.

10. API Layer & Supabase Client

The API layer replaces Flutter's Dio-based ApiClient and Supabase Flutter SDK with the Supabase JS SDK v2 and Next.js Server Actions. The architecture distinguishes three data access patterns: **Direct Supabase Queries** (for CRUD operations via PostgREST), **Server Actions** (for mutations with validation, authorization, and side effects), and **Edge Functions** (for operations requiring service role keys or external API calls). This replaces the Flutter pattern of RemoteDataSource -> Supabase SDK calls with a more type-safe, server-centric approach.

10.1 Data Access Pattern Mapping

Flutter Pattern	Next.js Pattern	When Used	Auth Context
AuthRemoteDataSource -> Supabase.auth.*	Server Action -> supabaseServer.auth.*	Login, signup, password reset	Server cookies
RemoteDataSource -> Supabase.from(table).select()	Server Component -> supabaseServer.from(table).select()	Read data for SSR	Server cookies
RemoteDataSource -> Supabase.from(table).insert()	Server Action -> supabaseServer.from(table).insert()	Create records	Server cookies
RemoteDataSource -> Supabase.from(table).update()	Server Action -> supabaseServer.from(table).update()	Update records	Server cookies
Supabase.channel().on()	Client hook -> supabaseBrowser.channel().on()	Realtime subscriptions	Browser client
Edge Function calls (ai-complete, etc.)	Edge Function calls (unchanged)	AI generation, payments	Service role key
Dio ApiClient (external APIs)	fetch() in Server Actions / Route Handlers	External API integration	Server-side only

10.2 Supabase Type Generation

Database types are auto-generated using `npx supabase gen types typescript` and stored in `lib/supabase/types.ts`. This provides compile-time type safety for all database queries, replacing the Flutter pattern of hand-written `UserModel.fromSupabaseUser()` and `fromJson/toJson` methods. The generated types include table row types, insert types, update types, and composite types for RPC function returns, ensuring that every Supabase query is fully typed without manual model definitions.

10.3 Edge Functions (Unchanged)

All Supabase Edge Functions remain unchanged from the Flutter implementation. They run on Deno at the edge and are invoked via the Supabase JS SDK's `invoke()` method from both Server Actions and Client Components. The shared utilities in `supabase/functions/_shared/` (`auth.ts`, `cors.ts`, `security_headers.ts`, `rate_limiter.ts`) continue to provide cross-function authentication, CORS handling, security headers, and rate limiting.

Edge Function	Purpose	Invoked From	Flutter Equivalent
ai-complete	AI question generation (non-streaming)	Server Action / Client	<code>AIService.generateQuestions()</code>
ai-stream	AI streaming generation	Client Component (SSE)	<code>AIService.stream()</code>
flutterwave-checkout	Payment checkout initialization	Server Action	<code>BillingRemoteDataSource.initCheckout()</code>
flutterwave-verify	Payment verification	Server Action	<code>BillingRemoteDataSource.verifyPayment()</code>
flutterwave-webhook	Payment webhook handler	Route Handler (POST)	N/A (server-side only)
process-refund	Refund processing	Server Action	<code>BillingRemoteDataSource.processRefund()</code>
health-check	Health check endpoint	Monitoring	N/A
marketplace-download	Resource download with quota	Server Action	<code>MarketplaceRemoteDataSource.download()</code>
exam-timing	Exam timing enforcement	Server Action	<code>ExamTimerService</code>

11. Server Actions & Server/Client Components

Server Actions replace the Flutter pattern of `UseCase -> Repository -> RemoteDataSource` with a single, cohesive server-side function that combines validation, authorization, data access, and side effects. They execute on the server without an explicit API endpoint, reducing network overhead and enabling progressive enhancement (forms work without JavaScript). The Server/Client Component boundary determines what code ships to the browser versus what stays on the server, directly impacting bundle size and performance.

11.1 Server Action Pattern

Every Server Action follows a consistent structure: **Zod input validation**, **Authorization check** (role + ownership), **Supabase operation**, **Side effects** (invalidation, notifications), and **Typed return** (`Result<T, E>`). This replaces the Flutter pattern where validation (`Formz`), authorization (scattered

role checks), data access (RemoteDataSource), and side effects (provider callbacks) were distributed across multiple layers.

Action Category	Example Actions	Validation Schema	Authorization
Auth	login, signup, logout, resetPassword, updateProfile	auth.loginSchema, auth.signupSchema	Public (login/signup) or Session (others)
Questions	createQuestion, updateQuestion, deleteQuestion, importQuestions	question.createSchema, question.updateSchema	Role: teacher+ for create/update
Exams	createExam, updateExam, submitAnswer, completeExam	exam.createSchema, exam.answerSchema	Role: teacher+ for create; student for submit
Billing	createSubscription, processPayment, cancelSubscription	billing.checkoutSchema	Session + ownership check
Marketplace	createProduct, purchaseProduct, submitReview	marketplace.productSchema	Role: teacher+ for create; student for purchase
School Mgmt	createSchool, updateSchool, manageTeachers	school.createSchema	Role: schoolAdmin+ only
Communication	sendMessage, createConversation, postToForum	communication.messageSchema	Session + membership check

11.2 Server vs Client Component Decision Matrix

The Server/Client Component boundary is the most critical architectural decision in Next.js. The principle is simple: **Server Component by default; Client Component only when the component requires event handlers, React hooks, browser APIs, or realtime subscriptions**. This table documents the decision for each major component category.

Component Type	Rendering	Rationale	Example
Data display pages	Server Component	Static data from Supabase, no interactivity needed	Dashboard pages, detail pages, list pages
List pages with filters	Server (data) + Client (filters)	Filters need useState/useSearchParams; data is SSR	Question list, exam list, marketplace search
Form pages	Client Component	Requires useForm, validation, controlled inputs	Login, register, create/edit forms
Data tables	Server (initial) + Client (sorting/pagination)	Interactive sort/page need hooks; initial data SSR	User management, billing history
Realtime components	Client Component	Requires Supabase Realtime subscription	Exam monitor, chat, live analytics
Interactive widgets	Client Component	Requires useState, useEffect, event handlers	Exam timer, anti-cheat, auto-save
Charts/graphs	Client Component	Requires Recharts which uses DOM	Analytics dashboard, result visualization
Layout components	Server (shell) + Client (interactive parts)	Sidebar/header need interactivity	App shell, navigation, theme toggle

12. Realtime Architecture

Supabase Realtime replaces the Flutter RealtimeService for live updates during exams, chat messaging, and collaborative features. The architecture uses a custom `useRealtime` hook that manages Supabase channel subscriptions with automatic cleanup, reconnection on network recovery, and integration with TanStack Query for cache updates. This replaces the Flutter pattern of managing channels manually in RealtimeService.dart with a declarative, hook-based approach.

12.1 Realtime Channels

Channel	Feature	Events	Hook	Flutter Equivalent
exam-session:{examId}	CBT Engine	postgres_changes (INSERT/UPDATE on answers)	useExamRealtime(examId)	RealtimeService (exam channel)
exam-monitor:{examId}	Exam Monitoring	presence (student join/leave), broadcast (progress)	useExamMonitor(examId)	RealtimeService (monitor channel)
chat:{conversationId}	Communication	postgres_changes (INSERT on messages)	useChatMessages(conversationId)	CommunicationProvider realtime
notifications:{userId}	Notifications	postgres_changes (INSERT on notifications)	useNotifications(userId)	NotificationService FCM
analytics:live	Dashboard	broadcast (live metrics)	useLiveAnalytics()	OptimizedRealtimeManager

12.2 Realtime Hook Pattern

The `useRealtime` hook provides a declarative API for subscribing to Supabase Realtime channels. It automatically handles channel creation, subscription, event handling, and cleanup on unmount. When a realtime event is received, it invalidates the relevant TanStack Query keys, triggering a background refetch to ensure cache consistency. For exam-taking scenarios, the hook also writes events to IndexedDB for offline recovery, mirroring the Flutter AutoSaveService pattern.

- **Connection management:** The hook uses Supabase Realtime's built-in reconnection with exponential backoff. Network status is monitored via the Navigator.onLine API and the useOffline hook, with automatic reconnection when connectivity is restored.
- **Presence tracking:** Exam monitoring uses Supabase Presence to track which students are online, when they joined, and their current question index. This replaces the Flutter RealtimeService presence tracking.
- **Broadcast:** Time-critical events (exam start/stop, timer sync) use Supabase Broadcast for low-latency peer-to-peer messaging, avoiding the database round-trip of postgres_changes.
- **Offline buffering:** When offline, realtime events are buffered in IndexedDB and replayed on reconnection. This ensures that exam progress is never lost, matching the Flutter SessionRecoveryService behavior.

13. Offline & IndexedDB Architecture

The offline architecture replaces Flutter's Drift (SQLite) local database with **Dexie.js**, a clean IndexedDB wrapper that provides a promise-based API, migration support, and relational queries. Offline support is critical for the CBT (Computer-Based Test) engine, where students must continue

taking exams even if network connectivity is lost. The architecture follows an "online-first with offline fallback" strategy: all operations attempt the server first, and transparently fall back to IndexedDB when the network is unavailable.

13.1 IndexedDB Schema

Table	Primary Key	Indexes	Purpose	Flutter Drift Equivalent
examSessions	examId	[examId+studentId], [status]	Active exam sessions with questions	LocalDatabase exam_sessions
examAnswers	[examId+questionId]	[examId], [syncStatus]	Student answers with auto-save	LocalDatabase exam_answers
questionCache	id	[subject+topic], [updatedAt]	Cached question bank for offline browse	CacheManager question cache
draftQuestions	id	[examId], [createdAt]	Draft questions being composed	N/A (new)
syncQueue	id	[syncStatus+createdAt]	Pending mutations for background sync	SyncEngine queue
userPreferences	key	None	Theme, layout, notification preferences	SharedPreferences + SecureStorage
conversationCache	id	[updatedAt]	Cached conversations for offline chat	N/A (new)

13.2 Sync Engine

The sync engine replaces Flutter's SyncEngine and AutoSaveService with a Service Worker-based background sync mechanism. When the user performs a mutation while offline (e.g., submitting an exam answer), the mutation is written to the IndexedDB syncQueue table with a status of "pending". When connectivity is restored, the Service Worker's sync event fires, processes the queue in order, and updates the sync status. This provides seamless offline-to-online transition without user intervention.

- **Background Sync API:** Uses the Service Worker Background Sync API (with Workbox fallback) to retry failed mutations when the network recovers. This is superior to the Flutter approach of polling connectivity with ConnectivityPlus.
- **Conflict resolution:** Uses a "server wins" strategy for most data. For exam answers, uses a "client wins" strategy since answers must never be lost. The server validates answer timestamps on sync to prevent cheating.
- **Progressive sync:** Large datasets (question bank, school data) use incremental sync with cursor-based pagination. Only changed records are fetched since the last sync timestamp.

14. Theme System & Design System

The theme system replaces Flutter's Material 3 ThemeNotifier with a CSS custom properties approach powered by **next-themes** and **Tailwind CSS v4**. The Flutter implementation used a ThemeNotifier Riverpod provider that stored ThemeMode + seed color in SharedPreferences, generating a full Material 3 ColorScheme at runtime via ColorScheme.fromSeed(). In Next.js, the same capability is achieved through CSS custom properties set on the root element, with Tailwind's dark mode variant handling theme switching.

14.1 Design Token Mapping

Flutter Token	Next.js Token	CSS Variable	Value (Light)	Value (Dark)
Brand Seed: Indigo 600	Primary	--primary	#4F46E5	#818CF8
ColorScheme.primary	Primary Foreground	--primary-foreground	FFFFFF	FFFFFF
ColorScheme.surface	Background	--background	FFFFFF	#0F172A
ColorScheme.onSurface	Foreground	--foreground	#1E293B	#F8FAFC
ColorScheme.surfaceVariant	Card	--card	#F1F5F9	#1E293B
ColorScheme.outline	Border	--border	#E2E8F0	#334155
Semantic Success	Success	--success	#16A34A	#22C55E
Semantic Warning	Warning	--warning	#F59E0B	#FBBF24
Semantic Error	Destructive	--destructive	#DC2626	#EF4444
Semantic Info	Info	--info	#2563EB	#60A5FA
Spacing xs=4	Spacing 1	--spacing-1	4px	4px
Spacing sm=8	Spacing 2	--spacing-2	8px	8px
Spacing md=12	Spacing 3	--spacing-3	12px	12px
Spacing lg=16	Spacing 4	--spacing-4	16px	16px
Spacing xl=24	Spacing 6	--spacing-6	24px	24px
Radius sm=8	Radius sm	--radius-sm	8px	8px
Radius md=12	Radius md	--radius-md	12px	12px
Radius lg=16	Radius lg	--radius-lg	16px	16px

14.2 Typography Scale

Flutter Style	Tailwind Class	Font Size	Weight	Line Height	Usage
displayLarge (57px)	text-6xl	3.75rem (60px)	700 (bold)	1.1	Hero numbers, splash screen
headlineLarge (32px)	text-4xl	2.25rem (36px)	700 (bold)	1.2	Page titles
headlineMedium (28px)	text-3xl	1.875rem (30px)	600 (semibold)	1.25	Section headings
titleLarge (22px)	text-2xl	1.5rem (24px)	600 (semibold)	1.3	Card titles
titleMedium (16px)	text-lg	1.125rem (18px)	500 (medium)	1.4	List titles, nav items
bodyLarge (16px)	text-base	1rem (16px)	400 (regular)	1.6	Body text
bodyMedium (14px)	text-sm	0.875rem (14px)	400 (regular)	1.5	Secondary text

Flutter Style	Tailwind Class	Font Size	Weight	Line Height	Usage
labelLarge (14px)	text-sm font-medium	0.875rem (14px)	500 (medium)	1.4	Button text, form labels
labelSmall (11px)	text-xs	0.75rem (12px)	400 (regular)	1.3	Captions, meta text
Custom: navLabel (12px)	text-xs	0.75rem (12px)	500 (medium)	1.2	Navigation labels

15. shadcn/ui Component Mapping

shadcn/ui provides copy-owned, Radix UI-primitive-based components styled with Tailwind CSS. Each Flutter widget from the `shared/widgets/` directory and feature-specific widgets maps to a shadcn/ui component or a custom composition. The copy-owned approach means components live in `components/ui/` and can be modified per-project without forking constraints, unlike traditional component libraries.

15.1 Shared Widget Mapping

Flutter Widget	shadcn/ui Component	Custom Composition	Notes
AppButton (ElevatedButton)	Button	No	Variants: default, destructive, outline, secondary, ghost, link
AppCard	Card	Card + CardHeader + CardContent + CardFooter	Standard composition pattern
AppDialog	Dialog	Dialog + DialogTrigger + DialogContent + DialogHeader	Replaces showDialog()
AppTextField	Input	Input + Label + FormMessage (RHF integration)	With React Hook Form + Zod
AppLoading	Skeleton	Skeleton	Pulse animation matches Flutter Shimmer
AppEmptyState	Custom	EmptyState component (icon + title + description + action)	Maps to <code>app_empty_state.dart</code>
AppErrorState	Custom	ErrorState component (error + retry + details)	Maps to <code>app_error_state.dart</code>
AppStatCard	Custom	StatCard (Card + icon + value + label + trend)	Maps to <code>app_stat_card.dart</code>
AppBar	Custom	Header (logo + search + notifications + user menu)	Maps to <code>app_app_bar.dart</code>
AppBottomNav	Custom	MobileNav (Sheet + navigation items)	Sheet for mobile bottom nav
AppNavigationDrawer	Custom	Sidebar (collapsible + role-filtered)	Maps to <code>app_navigation_drawer.dart</code>
AppSearchBar	Command	Command (cmdk) for search + command palette	Global search via Cmd+K

15.2 Additional shadcn/ui Components Required

Component	Usage	Feature
Avatar	User avatars in header, chat, profiles	All features
Badge	Status indicators, role badges, tags	Dashboard, exams, marketplace
Breadcrumb	Navigation breadcrumb in header	All authenticated pages
Calendar	Date picker for schedules, deadlines	Teacher workspace, communication
Checkbox	Multi-select questions, permissions	Question bank, admin
Collapsible	Sidebar sections, FAQ, accordion	Sidebar, help
Combobox	Subject/topic selectors with search	Question bank, exam builder
Data Table	Sortable, filterable, paginated tables	All management pages
Date Picker	Date range selection	Analytics, billing
Dropdown Menu	Context menus, row actions	All list pages
Form	React Hook Form integration	All form pages
Menubar	Top menu bar for admin	Super admin
Navigation Menu	Tab navigation within pages	Dashboard, settings
Popover	Contextual info, date pickers	Calendar, filters
Progress	Exam progress, upload progress	CBT engine, file upload
Radio Group	Single-choice questions (MCQ)	CBT engine
Select	Dropdown selectors	Filters, forms
Sheet	Side panels, mobile nav	Mobile, detail panels
Sonner (Toast)	Notifications, success/error messages	Global feedback
Switch	Toggle settings, feature flags	Settings, admin
Tabs	Tabbed content within pages	Dashboard, profile, settings
Textarea	Long text input (essay questions)	CBT engine, forms
Tooltip	Contextual help text	All features

16. Tailwind CSS Architecture

Tailwind CSS v4 replaces Flutter's Material 3 component theming with a utility-first approach augmented by CSS custom properties for theme tokens. The configuration extends the default theme with ExamForge's design tokens (colors, spacing, typography, radii) derived from the Flutter AppColors, AppTypography, and Spacings classes. Dark mode is handled via the `class` strategy with `next-themes`, providing SSR-safe theme switching without flash of unstyled content.

16.1 Custom Configuration

Config Section	Customization	Source Mapping
colors.primary	#4F46E5 (Indigo 600) with 50-950 scale	AppColors.brandSeedColor

Config Section	Customization	Source Mapping
colors.success/warning/error/info	Semantic color scale	AppColors semantic colors
colors.surface/card/muted	Surface hierarchy colors	AppColors surface colors
fontFamily.sans	Inter, system-ui, sans-serif	AppTypography (Inter family)
fontSize	Custom scale matching Flutter typography	AppTypography full M3 type scale
spacing	4px grid: 1=4px, 2=8px, ... 12=48px	Spacings class (4px grid system)
borderRadius	sm=8px, md=12px, lg=16px, xl=24px, full=9999px	AppColors borderRadius values
boxShadow	Custom elevation scale (sm=1, md=2, lg=4, xl=8)	AppColors elevation values
breakpoints	sm=640, md=768, lg=1024, xl=1280, 2xl=1536	ResponsiveFramework breakpoints

16.2 CSS Custom Properties for Theming

The root layout defines CSS custom properties that drive both Tailwind utility classes and shadcn/ui component styling. This enables dynamic theming (changing the seed color at runtime) by updating CSS variables on the root element, matching the Flutter ThemeNotifier's runtime seed color change capability.

16.3 Responsive Breakpoints

Breakpoint	Width	Layout	Navigation	Flutter Equivalent
sm	640px	Single column, full-width cards	Bottom navigation (MobileNav)	Mobile layout
md	768px	Two-column grid, side-by-side forms	Hamburger menu + sidebar	Tablet layout
lg	1024px	Sidebar + content, data tables	Persistent sidebar (expanded)	Desktop layout
xl	1280px	Wide content, multi-column dashboards	Persistent sidebar (expanded)	Large desktop
2xl	1536px	Max-width container (1400px), centered	Persistent sidebar (expanded)	Ultra-wide

17. Form & Validation Architecture

React Hook Form replaces Flutter's Formz for form state management, and Zod replaces the combination of Formz input validators and manual validation in UseCases. The architecture uses a **schema-first** approach where Zod schemas define both the validation rules and the TypeScript types (via z.infer), ensuring that the form structure, validation logic, and type definitions are always in sync. This eliminates the Flutter pattern of defining Formz input classes (EmailInput, PasswordInput, etc.) separately from the domain models.

17.1 Form Architecture Mapping

Flutter Pattern	Next.js Pattern	Benefit
Formz EmailInput	z.string().email()	Single source of truth for validation + type
Formz PasswordInput	z.string().min(8).regex(/[A-Z]/).regex(/[0-9]/)	Composable validation with clear error messages
Formz ConfirmPasswordInput	z.string().refine(val => val === password)	Cross-field validation via refine
Formz NameInput	z.string().min(2).max(100)	Simple schema with length bounds
AuthFormNotifier (StateNotifier)	useForm + zodResolver(schema)	Delegated validation, no manual state
TextEditingController	useForm register()	Uncontrolled by default (better perf)
FormKey + validate()	form.trigger() or submitHandler	Explicit validation on demand

17.2 Zod Schema Organization

Zod schemas are organized by feature in `lib/validators/` and within each feature module's `validators/` directory. Root schemas define shared primitives (email, password, uuid) that are composed into feature-specific schemas. Each Server Action imports and uses the same Zod schema as the client form, ensuring client and server validation are identical — a significant improvement over the Flutter pattern where client (Formz) and server (Edge Function) validation could diverge.

Schema File	Schemas	Used By
lib/validators/auth.ts	loginSchema, signupSchema, resetPasswordSchema, updateProfileSchema	Auth actions + forms
lib/validators/question.ts	createQuestionSchema, updateQuestionSchema, importSchema	Question bank actions + forms
lib/validators/exam.ts	createExamSchema, answerSchema, examSettingsSchema	CBT engine actions + forms
lib/validators/billing.ts	checkoutSchema, couponSchema, subscriptionSchema	Billing actions + forms
lib/validators/user.ts	updateUserSchema, roleAssignmentSchema	Admin actions + forms
features/*/validators/	Feature-specific schemas	Feature actions + forms

18. Error Boundary, Loading & Suspense Strategy

Next.js provides built-in conventions for error handling (`error.tsx`), loading states (`loading.tsx`), and streaming with Suspense boundaries. These replace the Flutter pattern of manual loading state management in StateNotifiers and the Sealed Result<T> type for error propagation. The architecture defines a consistent, hierarchical strategy for handling errors and loading states at every level of the route tree.

18.1 Error Boundary Hierarchy

Level	File	Scope	Behavior	Flutter Equivalent
Global	app/error.tsx	All routes	Full-page error with reset button, reports to Sentry	Try-catch in main.dart
Route Group	app/(authenticated)/error.tsx	All authenticated routes	Error within AppShell layout, preserves nav	Error in ShellRoute child
Feature	app/(authenticated)/exams/error.tsx	Exam feature only	Feature-specific error with contextual actions	Error state in ExamProvider
Page	app/(authenticated)/exams/[id]/error.tsx	Single exam page	Per-page error with retry, preserves URL	Error widget in page build()
Component	<ErrorBoundary> wrapper	Individual components	Component-level error, isolated from page	AppErrorState widget

18.2 Loading Strategy

Level	File	Rendering	Content	Flutter Equivalent
Global	app/loading.tsx	Server	Full-page skeleton (logo + spinner)	SplashPage
Route Group	app/(authenticated)/loading.tsx	Server	AppShell skeleton (sidebar + content area)	DashboardShell loading
Feature	app/(authenticated)/exams/loading.tsx	Server	Feature skeleton matching layout	Feature loading states
Page	app/(authenticated)/exams/[id]/loading.tsx	Server	Page skeleton matching final layout	Page loading state
Component	<Suspense fallback={...}>	Streaming	Inline skeleton for slow components	Shimmer widget
Mutation	TanStack Query isPending	Client	Button loading, optimistic UI	StateNotifier isLoading

18.3 Suspense Boundaries

Suspense boundaries enable **streaming SSR** — the page shell renders immediately while slow data-fetching components stream in as they resolve. This provides a dramatically faster perceived load time compared to Flutter's approach where the entire page must resolve before rendering anything. Key Suspense boundaries are placed around: dashboard stat cards (slow aggregation queries), data tables (potentially large datasets), chart components (Recharts requires client hydration), and AI-generated content (streaming responses from Edge Functions). Each Suspense boundary has a skeleton fallback that matches the final component layout, preventing layout shift when content resolves.

19. Feature Module Organization

Each feature module maps directly from its Flutter Clean Architecture counterpart, maintaining the domain/data/presentation separation while adapting to Next.js conventions. The key difference is that domain and data layers are collapsed into **Server Actions** (for mutations) and **Server Component queries** (for reads), while the presentation layer maps to **React components** and **custom hooks**. Each

feature exports a public API via its index.ts barrel file, enforcing encapsulation.

19.1 Feature Module Template

```
features/{feature-name}/
actions/ # Server Actions (mutations)
create.action.ts # "use server" — Zod validated, authorized
update.action.ts
delete.action.ts
{feature-specific}.action.ts
components/ # Feature-specific UI components
{feature}-card.tsx
{feature}-form.tsx
{feature}-detail.tsx
{feature}-list.tsx
hooks/ # Feature-specific hooks
use-{feature}.ts # TanStack Query + Supabase queries
use-{feature}-filters.ts
use-{feature}-realtime.ts # Realtime subscriptions
types/ # Feature-specific TypeScript types
index.ts # Generated from Supabase + custom
validators/ # Feature-specific Zod schemas
{feature}.ts # create/update/delete schemas
lib/ # Feature-specific pure logic
{feature}-utils.ts
{feature}-constants.ts
index.ts # Public API barrel export
```

19.2 Feature Inventory

Feature Module	Pag es	Server Actions	Client Hooks	Key Concerns
auth	5	5 (login, signup, logout, reset, verify)	useAuth, useSession	Cookie auth, PKCE, role enforcement
dashboard	5	1 (getDashboardData)	useDashboardStats	Role-specific data aggregation
question-ban k	8	6 (CRUD + import + export)	useQuestions, useQuestionFilters	Rich text editing, AI generation
ai-generator	7	4 (generate, validate, improve, stream)	useAiGeneration	SSE streaming, rate limiting
cbt-engine	8+	5 (create, update, submit, complete, recover)	useExam, useExamTimer, useAutoSave	Offline, anti-cheat, realtime
billing	11	6 (subscription, payment, refund, coupon, license)	useSubscription, usePayment	Flutterwave integration, webhooks
marketplace	14	5 (product CRUD, purchase, review)	useMarketplace, useCart	Search, filtering, seller dashboard
school-mgmt	30+	8 (school CRUD, teacher, student, class, subject)	useSchool, useTeachers	Complex CRUD, nested resources
teacher-works pace	16	10 (lesson, worksheet, assignment, etc.)	useWorkspace, useContentAssistant	AI content generation, templates
student-portal	11	4 (assignment, practice, goals, progress)	useStudentPortal, useAiTutor	AI tutoring, study planner
parent-portal	11	5 (child profile, messaging, insights)	useParentPortal	Child access authorization

Feature Module	Pag es	Server Actions	Client Hooks	Key Concerns
super-admin	12	6 (schools, users, billing, AI, security)	useAdminData	Service role access, PII handling
communicati on	13	5 (message, conversation, forum, announcement)	useChat, useRealtimeMessages	Realtime, presence
analytics	4	2 (platform, feature)	useAnalytics	Date range queries, aggregation
results	6	3 (view, grade, report)	useResults, useAiGrading	AI grading, PDF reports

20. Environment Variables & CI/CD

20.1 Environment Variables

Environment variables replace Flutter's flutter_dotenv pattern. Next.js provides built-in environment variable support with `.env.local`, `.env.production`, and `.env` files. Variables prefixed with `NEXT_PUBLIC_` are exposed to the browser; all others are server-only, accessible in Server Components, Server Actions, and Route Handlers. This is more secure than Flutter's approach where all `.env` values are bundled into the client build.

Variable	Exposure	Purpose	Flutter Equivalent
NEXT_PUBLIC_SUPABASE_URL	Public	Supabase project URL	SUPABASE_URL from .env
NEXT_PUBLIC_SUPABASE_ANON_KEY	Public	Supabase anonymous key	SUPABASE_ANON_KEY from .env
SUPABASE_SERVICE_ROLE_KEY	Server only	Admin operations (Edge Functions)	Not in client (server-only)
SUPABASE_JWT_SECRET	Server only	JWT verification in middleware	Not in Flutter client
NEXT_PUBLIC_APP_URL	Public	App base URL for redirects	APP_URL from app_config
FLUTTERWAVE_PUBLIC_KEY	Public	Flutterwave client-side encryption	FLUTTERWAVE_PUBLIC_KEY
FLUTTERWAVE_SECRET_KEY	Server only	Server-side payment operations	In Edge Functions only
OPENAI_API_KEY	Server only	AI generation (via Edge Functions)	In Edge Functions only
GEMINI_API_KEY	Server only	AI generation (via Edge Functions)	In Edge Functions only
NEXT_PUBLIC_SENTRY_DSN	Public	Sentry error reporting	Crash reporter config
SENTRY_AUTH_TOKEN	Server only	Sentry source map upload	Not in Flutter
NEXT_PUBLIC_GA_ID	Public	Google Analytics	Not in Flutter (new)

20.2 CI/CD Pipeline

Stage	Action	Tool	Trigger
Lint	ESLint + TypeScript type check	eslint + tsc --noEmit	Every commit
Unit Tests	Vitest with coverage	vitest --coverage	Every commit
Build	next build (type check + bundle)	Next.js build	Every PR
E2E Tests	Playwright browser tests	playwright test	PR to main
Preview Deploy	Deploy to Vercel preview	vercel deploy --build-env	Every PR
Production Deploy	Deploy to Vercel production	vercel deploy --prod	Merge to main
DB Migration	Apply Supabase migrations	supabase db push	Manual + CI gate
Type Gen	Regenerate Supabase types	supabase gen types	Schema change PR
Lighthouse	Performance, accessibility audit	lighthouse-ci	PR to main
Security Scan	Dependency audit + SAST	npm audit + semgrep	Weekly + PR

21. Testing Architecture

The testing architecture provides comprehensive coverage at every level: unit tests for pure logic and utilities, integration tests for Server Actions and Supabase queries, component tests for UI behavior, and end-to-end tests for critical user flows. This replaces the Flutter pattern of widget tests and integration tests, leveraging Next.js-specific tooling for testing Server Components and Server Actions that have no Flutter equivalent.

21.1 Testing Layers

Layer	Tool	Scope	What Tests	Flutter Equivalent
Unit	Vitest	Pure functions, utilities, Zod schemas, Zustand stores	Business logic, validation, state transitions	Unit tests with flutter_test
Component	Vitest + React Testing Library	React components in isolation	Rendering, user interactions, accessibility	Widget tests with flutter_test
Integration	Vitest + Supabase local	Server Actions with real DB	Data access, validation, authorization	Integration tests (manual)
API	Vitest + MSW	Route Handlers, Edge Functions	API contracts, error handling	API tests (manual)
E2E	Playwright	Full user flows in browser	Login, exam taking, payment, navigation	Integration tests (patrol/appium)
Visual	Playwright screenshots	Component visual regression	UI consistency across changes	Golden file tests

21.2 Critical E2E Test Scenarios

Scenario	Steps	Assertions	Priority
Student takes exam	Login -> Navigate to exam -> Answer questions -> Submit -> View result	Answers saved, timer works, result calculated	Critical
Teacher creates exam	Login -> Question bank -> Create questions -> Build exam -> Publish	Questions saved, exam visible to students	Critical
Payment flow	Login -> Billing -> Select plan -> Checkout -> Callback -> Verify	Subscription activated, invoice generated	Critical
Auth flow	Register -> Verify email -> Login -> Logout -> Login	Session maintained, redirects correct	Critical
Offline exam	Login -> Start exam -> Go offline -> Answer -> Go online -> Sync	Answers preserved, sync completes	High
Admin manages school	Login as admin -> Create school -> Assign teachers -> View dashboard	School created, teachers assigned	High
Realtime chat	Login -> Open conversation -> Send message -> Verify delivery	Message appears for both users	High
AI generation	Login -> AI Generator -> Configure -> Generate -> Review -> Save	Questions generated, validated, saved	Medium

22. Architecture Diagrams

This section provides textual descriptions of the key architecture diagrams that should be produced as visual artifacts during implementation planning. Each diagram is described with its nodes, edges, and grouping structure to enable precise rendering.

22.1 High-Level Architecture Diagram

Nodes: Browser Client (React RSC/CC) | Edge (Next.js Middleware) | Server (Next.js Server Components + Actions) | Supabase (Auth + PostgREST + Realtime + Storage) | Edge Functions (Deno) | External (OpenAI, Gemini, Flutterwave) | IndexedDB (Offline) | Service Worker (Background Sync). **Edges:** Browser -> Edge (HTTP request, middleware intercepts); Edge -> Server (authorized request); Server -> Supabase (cookie-authenticated queries/mutations); Browser -> Supabase (Realtime WebSocket); Server -> Edge Functions (invoke for AI/payments); Edge Functions -> External (API calls); Browser -> IndexedDB (offline reads/writes); Service Worker -> Server (background sync).

22.2 State Graph

Zustand Stores: auth-store (user, role, session) | ui-store (sidebar, search, toasts) | theme-store (mode, seed) | exam-session-store (activeExam, answers, timer) | notification-store (unread, toasts) | onboarding-store (step, complete). **TanStack Query Cache:** Per-feature query keys with stale/cache times. **Flow:** Server Action mutation -> TanStack Query invalidation -> Background refetch -> UI re-render. Auth state change -> Middleware cookie update -> Page revalidation.

22.3 Data Flow Diagram

Read path (SSR): User request -> Middleware (auth check) -> Server Component (Supabase query) -> Render HTML + dehydrate TanStack Query cache -> Stream to browser -> Hydrate client. **Read path (CSR navigation):** Link hover -> prefetchQuery (TanStack) -> Route transition -> Instant render from cache. **Write path:** User action -> Client Component (form submit) -> Server Action (validate +

authorize + mutate) -> Return result -> TanStack Query invalidate -> Background refetch -> UI update. **Realtime path:** Supabase DB change -> Realtime event -> useRealtime hook handler -> TanStack Query invalidate -> UI update. **Offline path:** Mutation while offline -> Write to IndexedDB syncQueue -> Service Worker sync event -> Replay queue to Server Actions -> Update sync status.

22.4 Dependency Graph

Core dependencies (shared): supabase-js -> all features; zod -> all Server Actions + forms; tanstack-query -> all data-fetching hooks; zustand -> all stores; next-themes -> root layout. **Feature dependencies:** cbt-engine -> auth, question-bank, results, realtime; billing -> auth, flutterwave (Edge Functions); marketplace -> auth, billing, storage; communication -> auth, realtime; analytics -> all features (read-only). **Cross-feature communication:** Via TanStack Query cache invalidation (no direct imports between features).

23. Migration Checklist

The following checklist provides a complete, ordered list of migration tasks organized by phase. Each phase is designed to be independently deployable, with earlier phases providing the foundation for later ones. No phase should begin until all items in the previous phase are complete and verified.

23.1 Phase 0: Foundation (Week 1-2)

- [] Initialize Next.js 16 project with App Router, TypeScript strict mode, Tailwind CSS v4
- [] Configure path aliases (@/, @/components/, @/lib/, @/features/)
- [] Set up Supabase JS SDK v2 with server/client/middleware clients
- [] Generate Supabase database types (npx supabase gen types typescript)
- [] Implement middleware.ts with auth, onboarding, and RBAC guards
- [] Configure next-themes with CSS custom properties for light/dark mode
- [] Set up Inter font via next/font/google with CSS variable
- [] Install and configure shadcn/ui CLI with project defaults
- [] Create root layout.tsx with ThemeProvider, QueryClientProvider, SupabaseProvider
- [] Create (public) layout.tsx and (authenticated) layout.tsx with AppShell
- [] Set up Zustand stores: auth-store, ui-store, theme-store
- [] Configure TanStack Query client with default stale/cache times
- [] Create lib/utils/cn.ts (className merger), lib/utils/result.ts (Result type)
- [] Set up ESLint, Prettier, Husky pre-commit hooks
- [] Create environment variable files (.env.local, .env.production)
- [] Verify: Unauthenticated routes redirect to /login, authenticated routes render AppShell

23.2 Phase 1: Authentication (Week 2-3)

- [] Implement auth Server Actions: login, signup, logout, resetPassword, verifyEmail
- [] Create Zod auth validators: loginSchema, signupSchema, resetPasswordSchema
- [] Build login page with React Hook Form + Zod + shadcn/ui form components
- [] Build register page with role enforcement (forced to student)
- [] Build forgot-password and reset-password pages

-
- [] Build verify-email page with deep link handling
 - [] Implement /auth/callback Route Handler for OAuth/Magic Link
 - [] Create useAuth hook consuming auth-store + TanStack Query
 - [] Implement onboarding flow with Framer Motion page transitions
 - [] Set up Sentry error reporting with Next.js integration
 - [] Verify: Full auth flow works (signup -> verify -> login -> logout -> login)
 - [] Verify: Middleware correctly guards all protected routes
 - [] Verify: Role-based redirects work for all 4 roles

23.3 Phase 2: Core Shell & Dashboard (Week 3-4)

- [] Build AppShell layout: Sidebar (collapsible, role-filtered), Header, MobileNav
- [] Implement sidebar navigation with role-based item filtering
- [] Implement header: user menu, notification bell, theme toggle, search (Command)
- [] Build role-specific dashboard pages (student, teacher, schoolAdmin, superAdmin)
- [] Implement dashboard stat cards with Server Component data fetching
- [] Create shared components: StatCard, EmptyState, ErrorState, LoadingSkeleton
- [] Build profile page and settings page
- [] Implement notification center with real-time updates
- [] Verify: Dashboard shows correct data for each role
- [] Verify: Sidebar navigation filters correctly per role
- [] Verify: Mobile layout works with bottom sheet navigation

23.4 Phase 3: Question Bank & AI Generator (Week 4-5)

- [] Build question bank pages: dashboard, list, create, detail, import, export, collections
- [] Implement question Server Actions with Zod validation
- [] Build rich text editor for question content (Tiptap)
- [] Implement question list with filters, sorting, and infinite scroll
- [] Build AI generator pages: generate, review, improve, document, history, prompts
- [] Implement AI streaming via SSE from ai-stream Edge Function
- [] Build AI review and improvement UI with validation engine
- [] Verify: Question CRUD works end-to-end
- [] Verify: AI generation streams responses correctly
- [] Verify: Question import/export maintains data integrity

23.5 Phase 4: CBT Engine & Offline (Week 5-7)

- [] Build exam list, create, detail, and edit pages
- [] Implement exam builder with question selection and configuration
- [] Build exam take page (Client Component) with timer, navigation, answer submission
- [] Implement useExamTimer hook with countdown and auto-submit
- [] Implement useAntiCheat hook (tab switch detection, copy prevention)
- [] Implement useAutoSave hook with periodic IndexedDB writes
- [] Set up Dexie.js IndexedDB schema for exam sessions and answers

-
- [] Implement Service Worker with Background Sync API
 - [] Build session recovery flow for interrupted exams
 - [] Build exam monitor page with Realtime (student presence, progress)
 - [] Build exam results and submission receipt pages
 - [] Implement exam templates feature
 - [] Verify: Student can take exam online and offline
 - [] Verify: Exam answers persist in IndexedDB and sync on reconnect
 - [] Verify: Timer auto-submits on expiry
 - [] Verify: Anti-cheat measures work correctly

23.6 Phase 5: Billing & Marketplace (Week 7-8)

- [] Build billing pages: plans, checkout, callback, history, invoices
- [] Implement Flutterwave checkout via Server Action -> Edge Function
- [] Handle payment webhook via Route Handler (POST /api/flutterwave-webhook)
- [] Build subscription management, AI credits, coupons, referrals pages
- [] Build marketplace pages: home, search, product detail, cart, seller dashboard
- [] Implement marketplace search with filters and pagination
- [] Implement purchase flow with quota management
- [] Verify: Payment flow works end-to-end (checkout -> pay -> callback -> verify)
- [] Verify: Marketplace browse and purchase work correctly

23.7 Phase 6: Portals & Communication (Week 8-10)

- [] Build school management pages (admin: schools, teachers, students, classes, subjects)
- [] Build teacher workspace pages (lesson plans, worksheets, assignments, rubrics, resources)
- [] Build student portal pages (assignments, flashcards, practice, goals, AI tutor)
- [] Build parent portal pages (child profiles, performance, attendance, messaging)
- [] Build super admin pages (schools, users, AI, billing, security, infrastructure)
- [] Build communication module: conversations, chat (realtime), forums, announcements, calendar
- [] Implement chat with Supabase Realtime (postgres_changes on messages)
- [] Build analytics dashboard with Recharts
- [] Build results module with AI grading and report generation
- [] Verify: All portal pages render with correct role-based access
- [] Verify: Chat messages appear in real-time for both users
- [] Verify: Analytics charts render with real data

23.8 Phase 7: Polish & Launch (Week 10-12)

- [] Implement Framer Motion page transitions and micro-interactions
- [] Add skeleton loading states for all pages
- [] Implement error boundaries at all levels (global, route group, feature, page)
- [] Run accessibility audit (Lighthouse + axe-core) and fix violations
- [] Implement progressive enhancement (core content works without JS)
- [] Optimize bundle size: dynamic imports for heavy components (Recharts, Tiptap, PDF)

-
- [] Configure Vercel deployment with edge middleware
 - [] Set up CI/CD pipeline (lint, test, build, preview deploy, production deploy)
 - [] Run Playwright E2E test suite for all critical user flows
 - [] Performance audit: Core Web Vitals (LCP, FID, CLS) under thresholds
 - [] Security audit: dependency scan, secret detection, CSRF verification
 - [] Load testing with k6 for concurrent exam sessions
 - [] Documentation: API reference, component storybook, architecture decision records
 - [] Verify: Lighthouse scores > 90 for performance, accessibility, best practices, SEO
 - [] Verify: All E2E tests pass in CI
 - [] Verify: Zero console errors in production build
-

MIGRATION SUMMARY

80+ screens mapped | 130+ routes documented | 100+ providers analyzed | 20+ features modularized |
7 phases planned | 12-week timeline | 0 production code written (specification only)